

Endless Mode

Surviving the context wipe
with a warm, clear memory.



Claude Code sessions don't just die—they get amnesia.



Separating the heavy traffic from the precious conversation.



Tool Traffic (80-95% of tokens)

Tool uses and results take up massive context.



The Conversation

User & Assistant messages are the thin, precious narrative.



The pieces for a **near-lossless, 10–20x smaller** record already exist. We already compress tool traffic in real time into tiny observations.

Forging the parallel transcript.



User Messages

Captured verbatim from the transcript. (The canonical source).

Assistant Messages

Captured verbatim. (Preserves the model's exact voice and plan).

Observations [#id]

Compressed one-line entries replacing massive tool traffic, tagged with lossless IDs for easy expansion via search.

Why rendering a derived view beats maintaining a file.



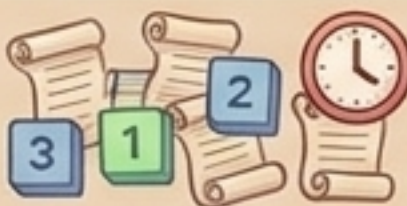
Continuous Append

Data Integrity



Dual-write drift and read-modify-write races.

Async Ordering



Out-of-order entries as observations land late.

System Impact



Write contention blocks the pipeline.

Future-Proofing

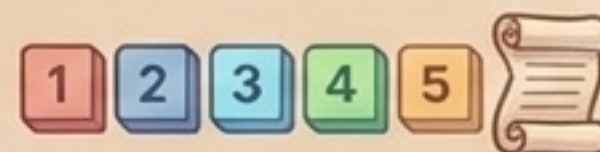


Locked into the format written at the time.

Derived View



Always consistent with canonical sources at seal time.



Backwards-looking merge-sort by position.



SQLite WAL reads add zero write contention.



Format upgrades retroactively regenerate history.

The render happens entirely on the worker island.



The Boss Fight: Surviving the mid-turn wipe.

Auto-Compact Wave

Observation count is variable by design! How do you know when I'm actually done?

pending_messages

pending_messages

The hardest part. Auto-compact usually fires mid-turn. New tool events might still be sitting in the pending queue, or inside an in-flight generation.

Measure the drain watermark, don't count the tools.



We **never** need to **know** how many observations should exist. We **only** need to know when the input has been fully processed.



A parseable SDK response clears the queue.



This signals: The agent is done with everything it was given.



Even if zero observations were extracted, the coverage is complete.

The Seal Operation decision tree.

PreCompact triggered.
Check Drain Status.

Drained? Yes ✓

Render instantly!
(gap: 0)

Drained? No ?

Wait up to 20s.
Crucial warning: Do NOT
interrupt the in-flight AI!

Timeout Hit?

Render an
Honest Tail.



Rendering the Honest Tail for un-processed events.



If we hit the timeout, we render stub lines for any tool **events** past the coverage point.

1: Narrative covered:

The verbatim assistant message already describes what it was doing.

2: Data survives:

The pending queue persists across worker restarts and lands in the DB minutes later.

3: Self-healing:

The bottle is a derived view. The next render automatically replaces stubs with real observations!

Threading the chronological clothesline

Observations land seconds late. To merge them perfectly, I thread an `anchor_timestamp` from the `tool event` to the pending row.



When storing a batch of observations, we stamp them with the batch's anchor range. This guarantees chronologically plausible placement for the merge-sort, rather than relying on misleading generation timestamps.

Maintaining identity across worker restarts.



The bottle is keyed
by the invariant
`contentSessionId`.

`memorySessionId`
A

`memorySessionId`
B

`memorySessionId`
C

Observations are keyed by `memorySessionId`, which changes on every worker restart. The render **MUST** gather observations across all mapped memory IDs, otherwise the bottle silently loses history.

The wake-up injection: read the bottle and continue.



- Pointer, not payload: We inject ~20 lines. The model pulls the full bottle through a Read tool.
- Self-cleaning: The bottle enters context as a tool result, which is exactly what the next compaction drops first. It never accumulates.
- Authoritative: Includes a load-bearing instruction to prevent the model from re-emitting finished work.

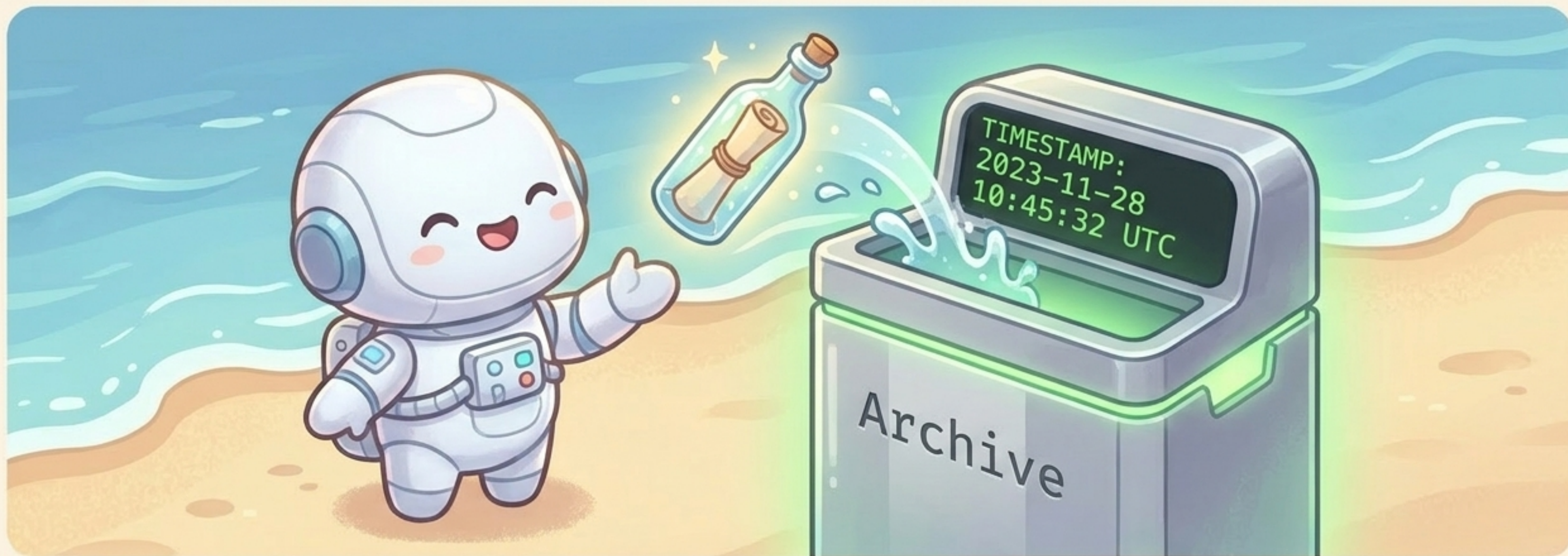
Graceful degradation means we never block the user.



The seal never blocks compaction. If anything fails, transport errors **exit cleanly** (exit 0) and the system silently falls back to today's timeline-only injection behavior.

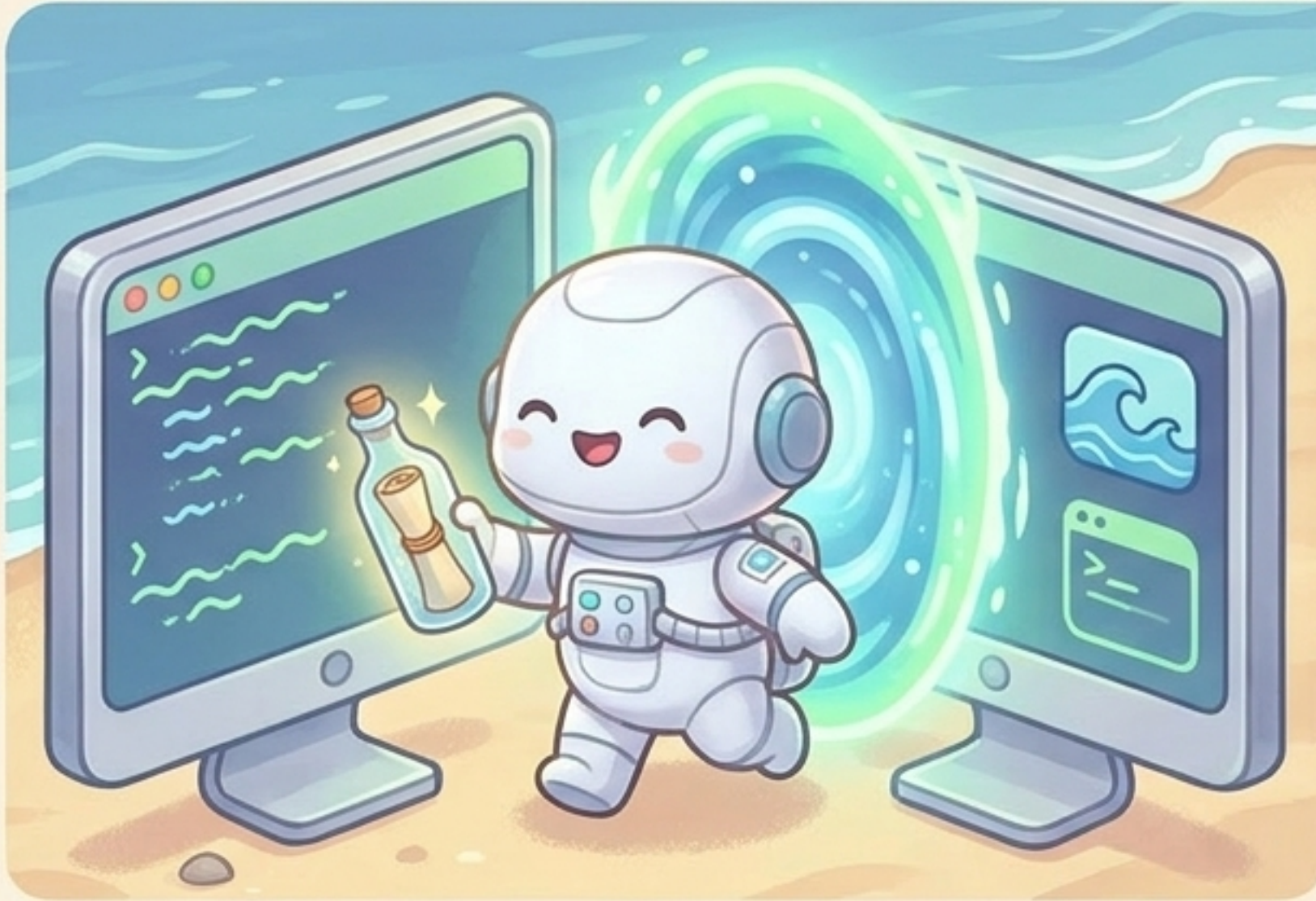
Endless Mode
degrades to
today's behavior,
never below it.

/Clear means forget.



When the user explicitly types `/clear`, the session starts fresh. The bottle is archived (via a timestamped move) alongside existing completion work, and nothing is injected.

The architectural dividends that fall out for free.



Session Portability: Because format upgrades are retroactive, you can render a bottle for any historical session. Resume any past session, anywhere.



Live Diary UX: A fire-and-forget render-on-Stop gives users a human-readable, always-current running record of their session on disk.

The endless journey.



Warm, clear, and truly endless.

- ✓ Survives amnesia via a near-lossless derived view.
- ✓ Solves async timing with the drain watermark and the honest tail.
- ✓ Maintains flawless chronological order via timestamp anchors.
- ✓ Points the fresh model at its own past with a single, self-cleaning instruction.